Introduction - Daniel

1. Project Title
    - Loan Approval Prediction
2. Team Member Names
    - Sandesh Manjunath Raykar
    - Fumiya Nagatomo
    - Vinay A Patil
    - Daniel Lee
3. Overall Context

Loan approval is an important process for banks and financial institutions, as they need to decide whether a person is likely to repay a loan or not. Traditionally, this decision is made by reviewing a person's financial and personal information, such as income, employment status, and credit history. However, this process can be slow and sometimes inconsistent.

With the help of data science and machine learning, it is possible to make this process faster and more reliable. By analyzing patterns in past loan applications, models can learn which factors are most important in determining whether a loan gets approved.

In this project, we use applicant and financial data to predict loan approval. The goal is not only to build a model that can make accurate predictions, but also to better understand which factors influence these decisions.

4. Related Work

Many previous projects and studies have treated loan approval prediction as a classification problem, where the goal is to predict whether a loan will be approved or rejected. Common machine learning models such as Logistic Regression, Decision Trees, and Random Forest are often used for this task.

Logistic Regression is usually used as a simple baseline model because it is easy to understand. Decision Trees and Random Forest models are more flexible and can capture more complex patterns in the data. Some studies also use more advanced models like Gradient Boosting to improve prediction performance.

In addition, past work has shown that factors like credit history, income level, and loan amount are very important in predicting loan approval. Recent studies also focus on making models more interpretable and fair, so that the results are easier to understand and do not unfairly affect certain groups.

This project follows similar approaches by testing multiple models and comparing their performance, while also trying to understand which features are the most important.

Proposed Solution

1. High-Level Framework

This project aims to develop a machine learning system that predicts whether a loan application will be approved or rejected based on applicant and financial information. The goal is not only to achieve accurate predictions but also to better understand the factors that influence lending decisions and provide explainable results. The project begins with data collection and preprocessing. The dataset includes features such as applicant income, co-applicant income, loan amount, loan term, employment information, and credit history. During preprocessing, missing values and inconsistencies are handled, categorical variables are encoded, and important features are prepared for modeling. After cleaning the data, exploratory data analysis (EDA) is performed to identify trends and relationships between variables and loan approval outcomes. Visualizations and correlation analysis help reveal how factors such as income level and credit history affect approval decisions. Several machine learning models are then trained and compared, including Logistic Regression, Decision Tree, Random Forest, and XGBoost. The models are evaluated using metrics such as accuracy, F1-score, and ROC-AUC to determine the most effective approach. Ensemble models such as Random Forest and XGBoost are expected to provide stronger performance because they can capture more complex patterns in the data. The project also focuses on explainability by analyzing feature importance and using SHAP to explain how different variables contribute to predictions. In addition, fairness considerations are discussed to examine whether prediction outcomes vary across demographic groups. Finally, the best-performing model is deployed using a Flask-based system that allows users to input applicant information and receive a prediction along with an explanation of the result. The final outcome includes the trained models, visualizations, presentation slides, GitHub repository, and a written report summarizing the project findings.

2. Uniqueness of the Solution

The uniqueness of this project comes from combining predictive machine learning with explainable and practical system design. While many loan prediction projects focus only on model accuracy, this project also emphasizes interpretability by using SHAP analysis to explain how different features influence approval decisions. This makes the system more transparent and suitable for real-world financial applications.

Another unique aspect is the deployment of the trained model through a Flask-based application. Instead of only presenting model results, the project provides a working demo that allows users to input applicant information and receive real-time predictions with explanations. In addition, the project compares multiple machine learning models, including Logistic Regression, Random Forest, and XGBoost, to evaluate both performance and interpretability. The inclusion of fairness considerations further strengthens the practical value of the system.

Methodology and Implementation - Vinay

1. Detailed Description of the Approach

The loan approval prediction system was implemented as a supervised binary classification pipeline, where each applicant record was classified as either approved or rejected. The first step was data preprocessing. The original dataset contained applicant financial and demographic information such as income, loan amount, loan term, CIBIL score, asset values, education status, and self-employment status. Since machine learning models require numerical input, categorical variables such as education and self-employment were converted into numerical features using one-hot encoding.

After encoding, redundant dummy columns were removed to avoid duplicate information and multicollinearity. The loan_id column was also removed because it only identifies each record and does not provide useful predictive information. The target variable was converted into a binary column, where approved loans were represented as 1 and rejected loans as 0. The final feature set contained 11 input variables, including cibil_score, loan_amount, loan_term, income, asset values, education status, self-employment status, and number of dependents.

The data was then split into training and testing sets using an 80/20 split. The training set was used to fit the models, while the test set was kept separate for evaluating model performance on unseen data.

Multiple models were implemented and compared, including Logistic Regression, Random Forest, and XGBoost. Logistic Regression was used as a baseline model, while Random Forest and XGBoost were selected because tree-based models can capture non-linear relationships more effectively. This was important because loan approval decisions were strongly affected by threshold-based patterns, especially around the CIBIL score.

XGBoost was implemented as an advanced gradient boosting model using 200 estimators, a maximum tree depth of 4, a learning rate of 0.05, and subsampling to reduce overfitting. Random Forest was also trained and later selected as the final model because it slightly outperformed XGBoost in both accuracy and ROC-AUC while still being easier to interpret. The models were evaluated using accuracy, ROC-AUC, precision, recall, F1-score, confusion matrix, and ROC curve.

To make the model more explainable, SHAP analysis was added. SHAP values helped identify how much each feature contributed to the model's prediction. This made it possible to explain not only whether an applicant was approved or rejected, but also which features influenced that decision. The explainability results showed that cibil_score was the dominant feature, followed by loan_term and loan_amount. Finally, the trained Random Forest model and feature column order were saved using Python's pickle module so that the same model could be loaded later in the Flask application for real-time loan approval prediction.

2. Implementation Summary

The implementation started by loading the loan approval dataset into a Pandas DataFrame. Categorical columns were transformed using one-hot encoding, and unnecessary or redundant columns were dropped. The feature matrix X was created by removing the loan_id and target column, while the target vector y used the encoded loan approval status.

The processed dataset was split into 3,415 training records and 854 testing records. The final input features used for training were:

no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, residential_assets_value, commercial_assets_value, luxury_assets_value, bank_asset_value, education_Graduate, and self_employed_No.

After preprocessing, XGBoost and Random Forest models were trained and compared. XGBoost achieved an accuracy of 0.9778 and ROC-AUC of 0.9981. Random Forest achieved an accuracy of 0.9789 and ROC-AUC of 0.9987, making it the strongest model overall. Because Random Forest performed slightly better and was easier to interpret using feature importance and SHAP, it was chosen as the final model.

The final model was evaluated using a confusion matrix, classification report, ROC curve, prediction probability distribution, and feature importance analysis. SHAP was used to explain the model's decisions at both global and individual applicant levels. The implementation was completed by saving the trained Random Forest model as rf_loan_model.pkl and saving the feature list as feature_names.pkl. Saving the feature names was important because the Flask API must receive applicant inputs in the same order used during training.

3. Key Code Snippets

Preprocessing and train-test split

```
df_onehot = pd.get_dummies(df)

df_onehot.drop(columns=[
    ' loan_status_ Rejected',
    ' self_employed_ Yes',
    ' education_ Not Graduate'
], inplace=True)

df_onehot.rename(columns=columns, inplace=True)

X = df_onehot.drop(columns=['loan_id', 'loan_status_Approved'])
y = df_onehot['loan_status_Approved']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape :", X_test.shape)
print("Features   :", list(X.columns))
```

XGBoost model training

```
xgb_model = xgb.XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
)

xgb_model.fit(
    X_train,
    y_train,
    eval_set=[(X_test, y_test)],
    verbose=False
)

y_pred = xgb_model.predict(X_test)
y_pred_prob = xgb_model.predict_proba(X_test)[:, 1]

print(f"XGBoost Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"ROC-AUC Score    : {roc_auc_score(y_test, y_pred_prob):.4f}")
```

Random Forest final model

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)
rf_pred_prob = rf.predict_proba(X_test)[:, 1]

print(f"Accuracy  : {accuracy_score(y_test, rf_pred):.4f}")
print(f"ROC-AUC   : {roc_auc_score(y_test, rf_pred_prob):.4f}")
print(f"Precision : {precision_score(y_test, rf_pred):.4f}")
print(f"Recall    : {recall_score(y_test, rf_pred):.4f}")
print(f"F1 Score  : {f1_score(y_test, rf_pred):.4f}")
```

SHAP explainability

```
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test)

if isinstance(shap_values, list):
    sv = shap_values[1]
else:
    sv = shap_values[:, :, 1]

shap.summary_plot(
    sv,
    X_test,
    plot_type="dot",
    show=False
)

plt.title("SHAP Summary - Feature Impact on Loan Approval (RF)")
plt.tight_layout()
plt.savefig("rf_shap_summary.png", dpi=150, bbox_inches="tight")
```

Experiments and Results - Sandesh

1. Experimental Setup
    - The experiments were conducted on the Loan Approval Prediction dataset, which contains 4,269 applicant records with 12 features, including demographic, financial, and asset-related attributes. The target variable is binary - loan approved (1) or rejected (0).

Pre-processing steps:

- Categorical variables (education, self_employed) were one-hot encoded, retaining one column per variable (e.g., education_graduate, self_employed_no) to avoid multicollinearity.
- The loan_id column was dropped as it carries no predictive information.
- The final feature set used for training comprised 11 features: number_of_dependants, income_annum, loan_amount, loan_term, cibil_score, residential_assets_value, commercial_assets_value, luxury_assets_value, bank_asset_value, education_graduate, and self_employed_no.

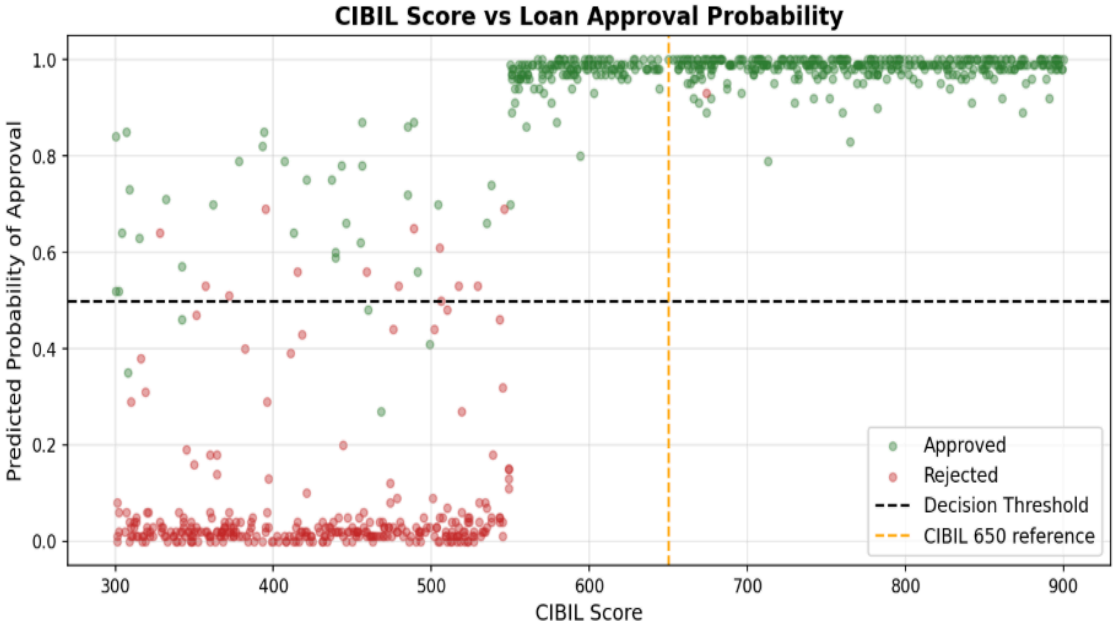
Train/Test Split: An 80/20 stratified split was applied, resulting in 3,415 training samples and 854 test samples.

Three classifiers were evaluated as part of this study:

- Logistic Regression - as a linear baseline
- Random Forest (100 estimators, default hyperparameters) - as the primary ensemble model
- XGBoost (200 estimators, max_depth=4, learning_rate=0.05, subsample=0.8) - as an advanced gradient boosting baseline

The trained Random Forest model was serialized using Python's pickle module and deployed via a Flask REST API (app.py), which accepts applicant data as JSON and returns a prediction with approval/rejection probabilities.

2. Results from Earlier Stages
    - Before the ensemble models were applied, Logistic Regression was used as a baseline. It achieved an accuracy of approximately 0.7295 and an ROC-AUC of 0.8032, indicating that a linear decision boundary is insufficient to capture the non-linear relationships in this dataset - particularly around the CIBIL score threshold.
    - The CIBIL Score vs Loan Approval Probability plot (Figure below) illustrates this clearly. Applicants with a CIBIL score above 650 are overwhelmingly approved, and the model assigns them near-certainty probabilities (close to 1.0). Below 650, the predictions are more mixed but still largely correct, confirming that CIBIL score is the dominant signal. A small number of high-scoring applicants are rejected and vice versa, attributable to their loan amount or term.

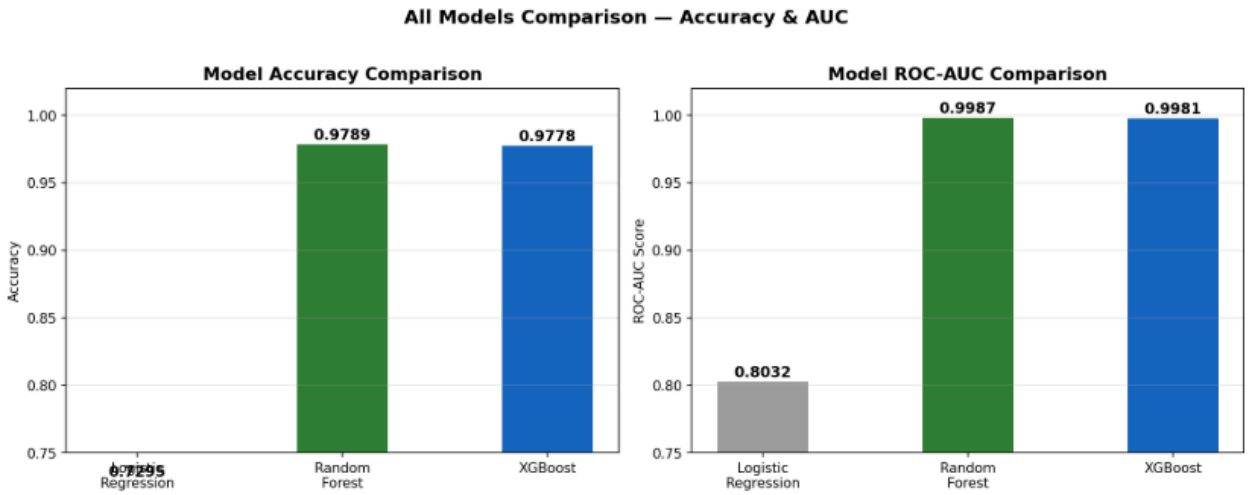


- These observations from the earlier Logistic Regression stage motivated the move to tree-based ensemble methods, which can naturally learn non-linear thresholds and feature interactions without manual feature engineering.

3. Comparison with Baseline Methods

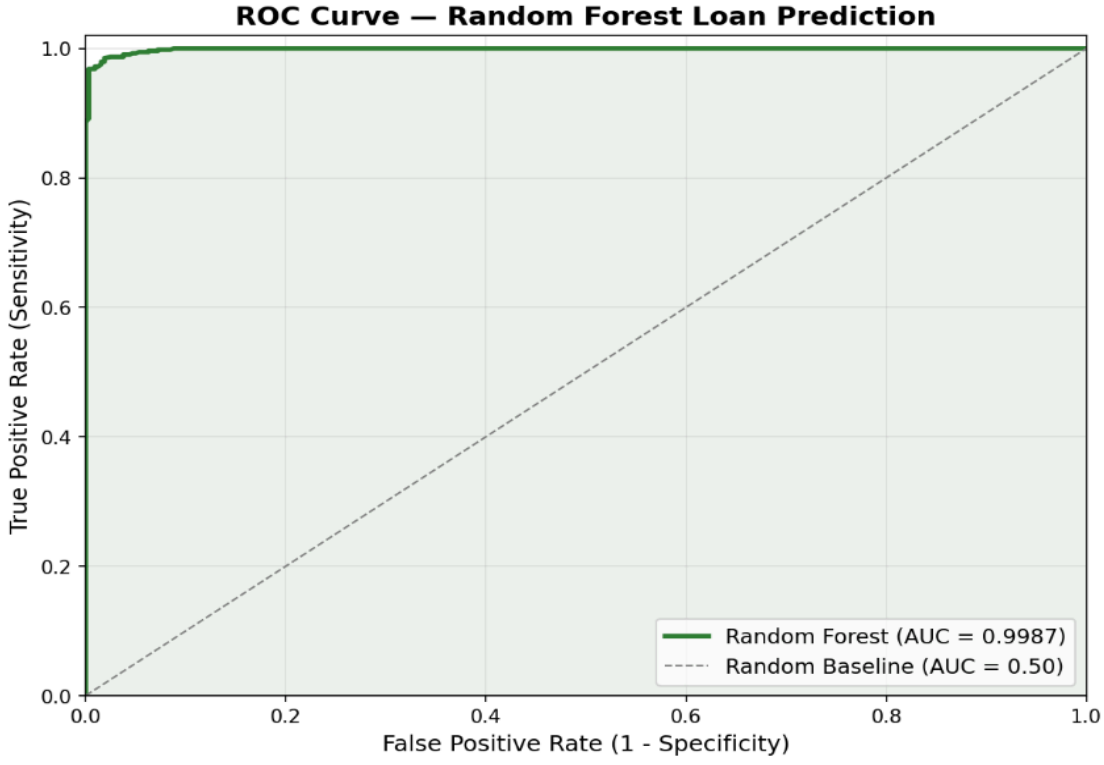
- The table below summarises performance across all three models on the held-out test set (854 samples):

Model	Accuracy	ROC-AUC
Logistic Regression	0.7295	0.8032
Random Forest	**0.9789**	**0.9987**
XGBoost	0.9778	0.9981



Random Forest marginally outperformed XGBoost on both metrics and was selected as the final deployed model. The gap between the linear baseline and both ensemble methods is substantial - roughly 25 percentage points in accuracy and 20 points in AUC - demonstrating that the decision boundary in this problem is inherently non-linear.

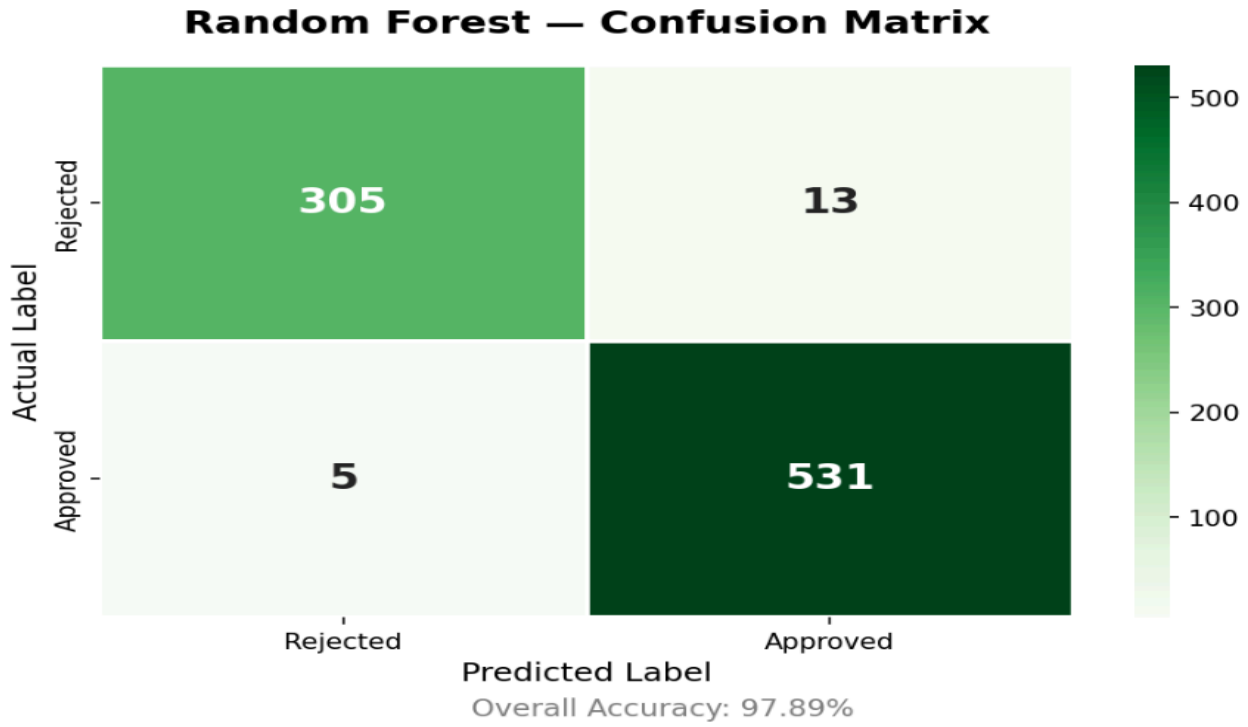
The ROC Curve for the Random Forest (AUC = 0.9987) shows the classifier reaches near-perfect sensitivity at an extremely low false positive rate, meaning it correctly identifies almost all approved applicants while raising very few false alarms.



4. Performance Analysis

Confusion Matrix (Random Forest, test set):

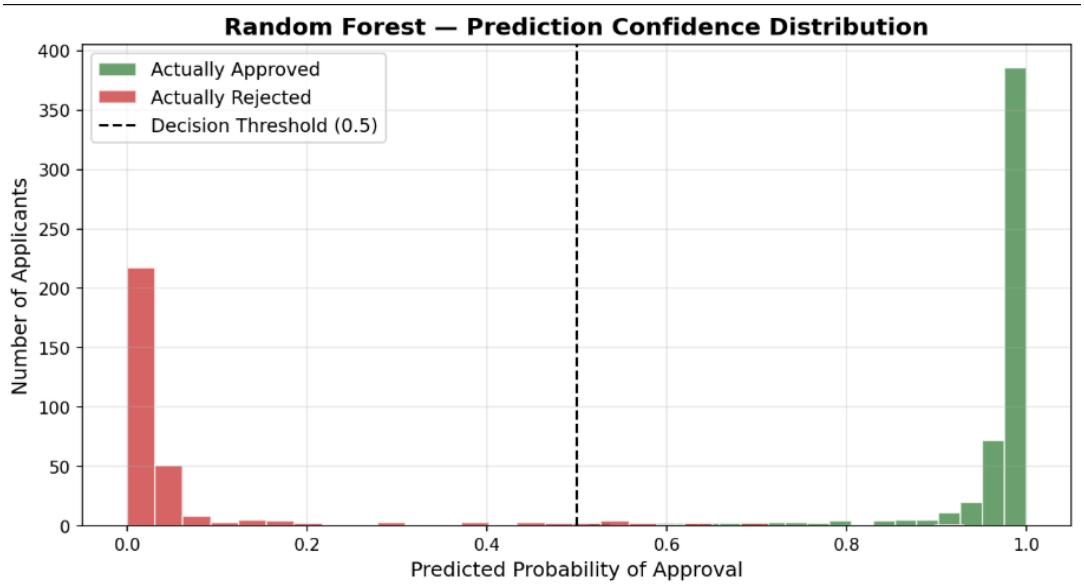
	Predicted Rejected	Predicted Approved
Actual Rejected	305 (TN)	13 (FP)
Actual Approved	5 (FN)	531 (TP)



- Overall Accuracy: 97.89%
- Precision (Approved): 97.61% - of all predicted approvals, 97.6% were genuinely creditworthy.
- Recall (Approved): 99.07% - the model correctly identified 99% of all actual approved applicants, minimising missed approvals.
- F1 Score (Approved): 98.33%
- Precision/Recall (Rejected): 98% / 96%, showing the model is also highly reliable when flagging rejections.

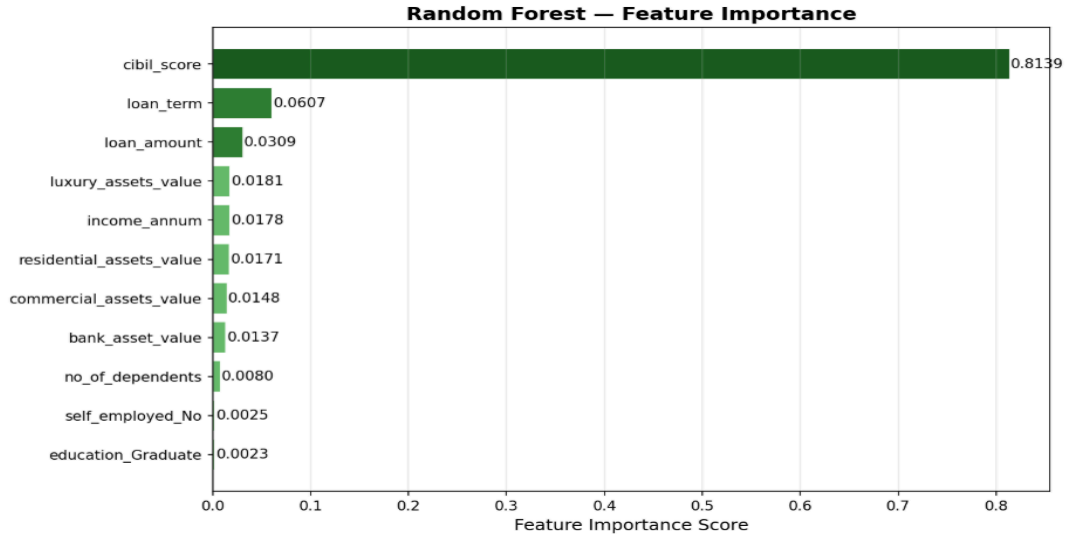
Prediction Confidence Distribution: The histogram of predicted probabilities shows a strongly bimodal distribution - the vast majority of predictions cluster near 0 (high-confidence rejections)

or near 1 (high-confidence approvals), with very few cases in the ambiguous 0.3-0.7 range. This indicates the model is decisive and well-calibrated on this dataset.

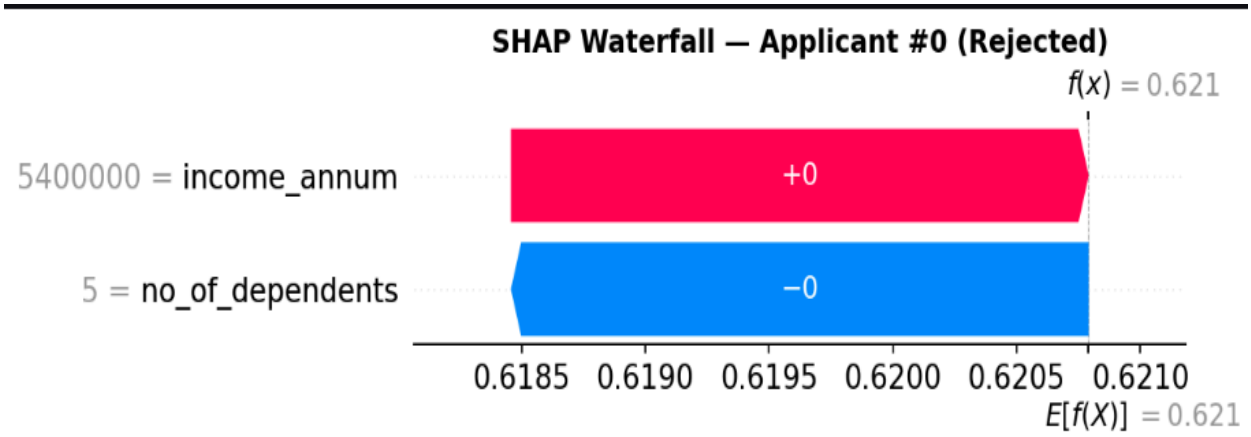


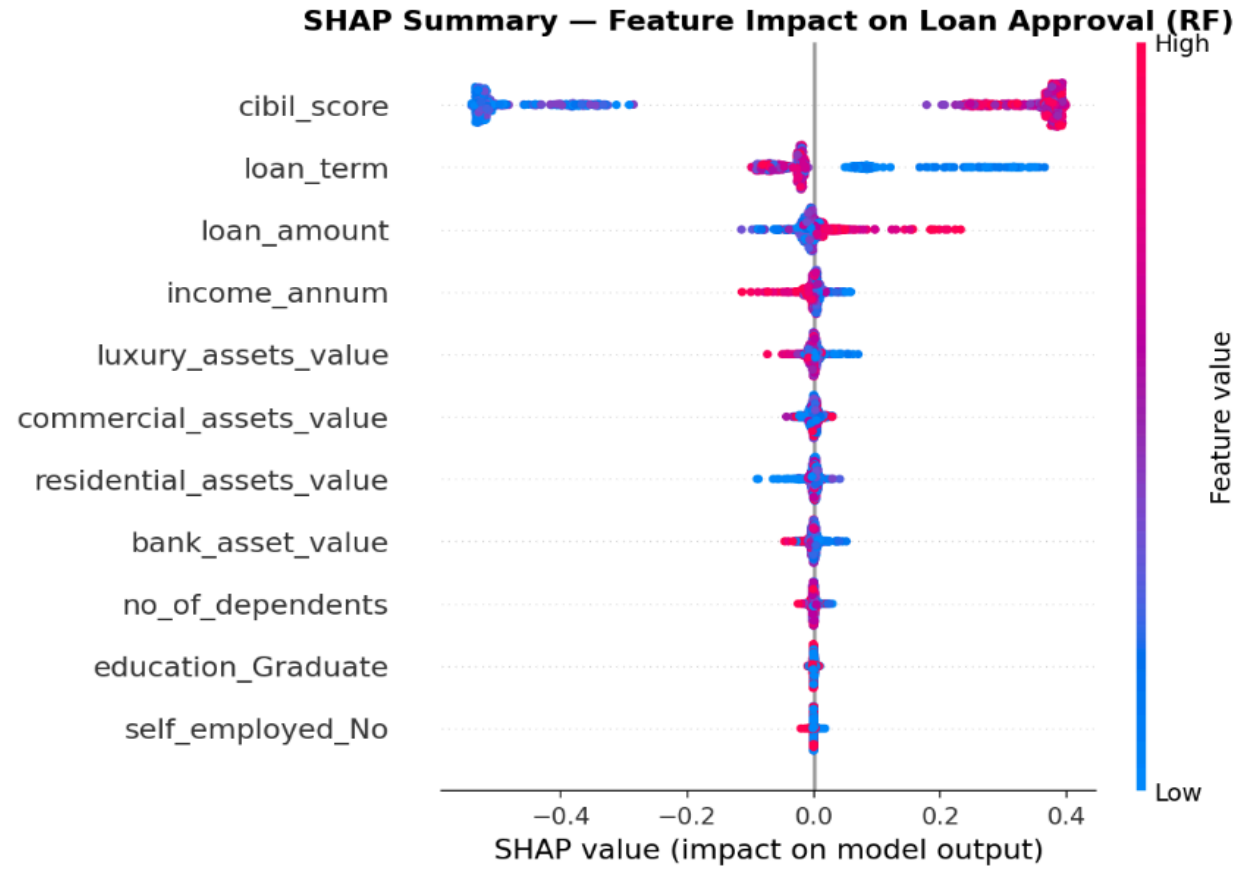
Feature Importance Analysis (Random Forest & SHAP):

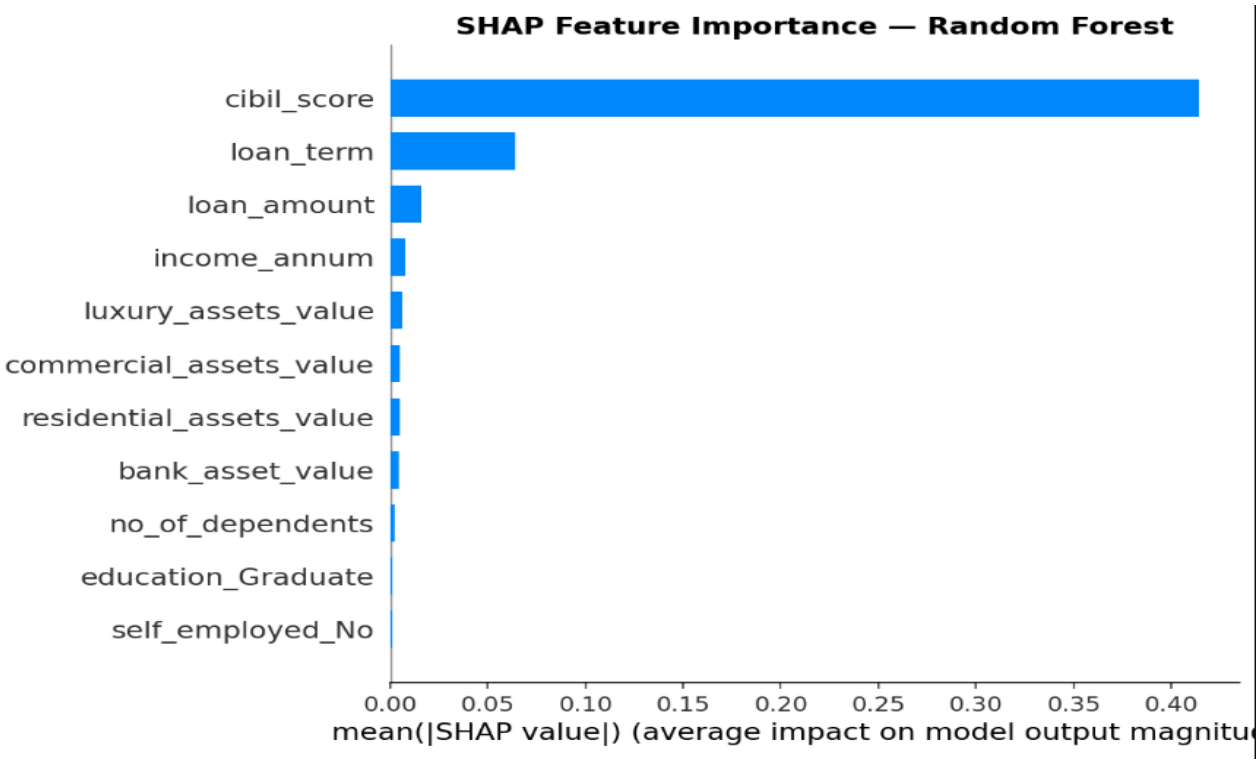
Both the native Random Forest feature importance scores and SHAP (SHapley Additive exPlanations) values converge on the same conclusion: cibil_score is overwhelmingly the most important predictor (importance score: 0.8139; mean |SHAP| ≈ 0.41), followed distantly by loan_term(0.0607) and loan_amount(0.0309). Asset-related features (luxury, residential, commercial, bank) and demographic features (education, employment status, dependents) contribute marginally but consistently.



The SHAP Summary plot further reveals directionality: high CIBIL scores push predictions strongly toward approval (positive SHAP values), while low CIBIL scores drive rejection (large negative SHAP values). For loan_term, longer terms mildly increase the approval probability, possibly reflecting the model's recognition that longer-term loans are associated with lower monthly burden relative to income. The SHAP Waterfall for a sample rejected applicant confirms these findings - the model's output is anchored almost entirely by the CIBIL score, with all other features contributing negligible marginal impact.







Discussion - Daniel

1. What Worked Well

- One of the most successful parts of this project was that the machine learning models were able to predict loan approval outcomes with relatively high accuracy. The dataset included financial information such as applicant income, loan amount, loan term, asset values, and CIBIL score, which allowed the models to learn patterns related to loan approval and rejection.

  In particular, tree-based models such as Random Forest and XGBoost performed very well. The project initially started with Logistic Regression as a baseline model. While it could capture some general patterns, it struggled to represent more complex relationships in the data. Random Forest, on the other hand, was able to handle non-linear relationships more effectively and achieved much higher performance.

  Another positive aspect of the project was the ability to identify which features had the greatest impact on loan approval decisions. The analysis showed that the CIBIL score was the most important factor, while features such as loan

amount and loan term also contributed to the predictions. Using SHAP analysis helped make the model outputs more understandable by showing how different variables influenced each prediction.

In addition, the demo application created by Sandesh helped demonstrate how the trained model could actually be applied in a practical setting. Instead of only showing model accuracy and evaluation results, the Flask-based application allowed users to enter applicant information and immediately receive a loan approval prediction with probabilities. This made the project feel more realistic and showed how machine learning models can be integrated into an interactive system similar to real financial applications.

2. What Did Not Work
    - There were also some limitations in this project. One of the biggest issues was that the dataset did not fully reflect real-world financial environments. The dataset was relatively simple and mainly designed for learning purposes, so it did not include many of the complex situations and detailed customer information that actual banks would use during the loan approval process.

      Because of this, the model sometimes produced overly extreme results. In particular, the CIBIL score had a very strong influence on the predictions. Applicants with high scores were almost always approved, while applicants with low scores were usually rejected. As a result, the model relied heavily on this feature, and many prediction probabilities were pushed very close to 0 or 1 instead of showing more balanced or uncertain outcomes.

      In addition, although the model achieved very high accuracy, the simplicity of the dataset likely made the prediction task easier than it would be in a real financial setting. Therefore, while the project was useful for demonstrating how machine learning can be applied to loan approval prediction, the results may not directly generalize to real-world banking systems without more realistic and diverse data.

3. Conclusion and Future Work
    - Through this project, we were able to directly experience the process of using machine learning models to predict loan approval outcomes. After comparing multiple models, Random Forest showed the best overall performance, and SHAP

analysis helped us understand which features had the biggest impact on predictions. For our group, it was interesting not only to improve the accuracy of the models, but also to better understand why the models made certain decisions.

One of the most impressive parts of the project was the Flask-based demo application created by Sandesh. It made the project feel much more like a real service rather than just a model experiment. Users could enter applicant information and immediately receive loan approval predictions and probabilities, which helped demonstrate how machine learning models could actually be applied in financial systems.

In the future, we would like to further develop this system into something closer to a real financial application. While the current model was mainly trained around factors such as CIBIL score, adding features more relevant to the U.S. financial system, such as credit score, loan history, and spending patterns, could help make the model more optimized for the American market. We also think improving the demo application with features such as personalized analysis or automatic reporting could make the system feel even closer to something that could be used in real banking or financial services.

Link to Github repo

Github